

Software Metrics

Testing is not a final phase.
It is a design, feedback and confidence mechanism.

Course block

LOC, code coverage, Coupling,
Cohesion, tools

Pedro García López · TAP

Software Metrics

Measuring software quality, design, testing and evolution

- Advanced Programming Techniques
- Code metrics · design metrics · testing metrics · process metrics
- With Java, GitHub Actions, coverage and AI-assisted development

metrics

complexity
coverage
coupling
cohesion
quality gates

2026

Why Measure Software?

Metrics help us discuss code quality with evidence

- Make invisible properties visible: complexity, coupling, duplication, testability
- Detect risky modules before they become maintenance problems
- Compare versions of the same codebase after refactoring
- Support engineering decisions without replacing human judgment

A metric is a signal, not a verdict.

What Can We Measure?

Different questions require different metrics

Product metrics

Size, complexity, coupling, cohesion, duplication, maintainability

Process metrics

Lead time, review time, change failure rate, defect escape rate

Testing metrics

Coverage, mutation score, flaky tests, assertion quality

Operational metrics

Latency, error rate, resource usage, reliability

In this chapter we focus mainly on product and testing metrics, but modern teams connect them with delivery metrics.

Metrics Are Not Objectives

Goodhart's law in software engineering

```
// A metric can be gamed
if (coverage < 90) {
    writeTestsThatOnlyExecuteCode();
}

// Better question:
// Which important behaviours are protected by tests?
```

- High coverage does not guarantee correctness
- Low complexity does not guarantee a good design
- More commits do not mean more productivity
- Metrics should trigger questions, reviews and refactorings

Size Metrics

Simple, useful, and dangerous when used alone

```
class ReportService {  
    Report generate(String source, Format format,  
                    boolean includeSummary,  
                    boolean anonymize,  
                    Locale locale) { ... }  
}  
  
// Many parameters may indicate a missing object.
```

- Lines of code: rough proxy for size and maintenance effort
- Number of classes, methods and packages: structure and modularity
- Number of parameters: possible smell in API design
- Number of public methods: size of the exposed contract

Cyclomatic Complexity

How many independent paths exist through a method?

- Approximated in practice by counting decisions: if, while, for, case, catch, &&, ||
- Higher complexity means more paths to understand and test
- Very complex methods are candidates for Extract Method or polymorphism
- A common threshold is to inspect methods above 10, not automatically reject them

$$M = E - N + 2P$$
$$M = P + 1$$

Complexity Example

Small business rules already create branches

```
public double calculateAppraisal(EmployeeDetails employee) {  
    if (employee.getMonthlySalary() < 10000) {  
        return 500;  
    } else {  
        return 1000;  
    }  
}  
  
// Paths: low salary, high salary  
// Minimum useful tests: both branches
```

- This TAP example has one decision point
- One test is not enough to validate both behaviours
- Metrics connect design and testing: each branch represents expected behaviour

Complexity Smell: Nested Decisions

When the method becomes the design

```
double discount(Customer c, Order o) {  
    if (c.isVip()) {  
        if (o.total() > 1000) return 0.20;  
        return 0.10;  
    }  
    if (c.hasCoupon()) return 0.05;  
    return 0.0;  
}
```

- Nested conditionals mix policy, data access and business rules
- Test cases grow quickly as branches combine
- Possible refactorings: Extract Method, Replace Conditional with Polymorphism, Strategy

Coupling

How strongly does one module depend on another?

```
class InvoiceService {  
    private final MySQLInvoiceRepository repo = new  
MySQLInvoiceRepository();  
    private final SmtEmailSender email = new  
SmtEmailSender();  
}  
  
// Concrete dependencies make testing and replacement harder.
```

- High coupling increases the cost of change
- Direct construction of dependencies hides design decisions
- Prefer interfaces and dependency injection for volatile collaborators
- Useful metrics: efferent coupling, afferent coupling, instability

Cohesion

Do the elements of a class belong together?

```
class StudentManager {  
    void enroll(Student s) { ... }  
    void calculateAverageGrade(Student s) { ... }  
    void exportPdf(Student s) { ... }  
    void sendEmail(Student s) { ... }  
}  
  
// One name, many responsibilities.
```

- Low cohesion usually means a class has several reasons to change
- Symptoms: unrelated fields, unrelated methods, vague names like Manager or Helper
- Refactorings: Extract Class, Move Method, introduce domain services

Duplication

Copy-paste is a design metric

```
double yearlySalary(EmployeeDetails e) {  
    return e.getMonthlySalary() * 12;  
}  
  
double yearlyTaxBase(EmployeeDetails e) {  
    return e.getMonthlySalary() * 12;  
}  
  
// Same expression, maybe same concept.
```

- Duplicated code increases defect probability and change cost
- Duplicated concepts can be more dangerous than duplicated lines
- Refactorings: Extract Method, Pull Up Method, Template Method, shared domain concept

Maintainability Index

A composite score for code maintainability

- Combines signals such as size, complexity and comment density
- Useful for spotting modules that deserve inspection
- Not a universal truth: formulas vary between tools
- Use trends over time rather than isolated absolute values

Best use: compare the same project before and after refactoring.

Technical Debt

A metaphor for future cost

```
// TODO: temporary workaround
// FIXME: duplicated validation
// HACK: assumes only one tenant

// Comments like these are debt signals.
```

- Debt is not always bad: sometimes teams deliberately take shortcuts
- The problem is unmanaged debt: hidden, untracked, and accumulating interest
- Metrics can reveal hotspots where debt is concentrated
- Refactoring should be justified by risk, frequency of change, and business value

Testing Metrics

Coverage is useful, but not sufficient

Line coverage

Was this line executed by tests?

Branch coverage

Were both sides of a decision executed?

Mutation score

Would tests fail if code were slightly changed?

Flaky test rate

How often do tests fail without code changes?

Coverage measures execution, not correctness. Assertions and scenarios matter.

Coverage Example

The calculator has simple line coverage

```
@Test
public void testDivide() {
    double value1 = 9;
    double value2 = 12;
    double result = value1 / value2;

    Calculator c = new Calculator();
    Assert.assertTrue(c.divide(value1, value2) == result);
}
```

- This executes the divide method
- But it does not test division by zero or precision concerns
- Coverage tells us what ran; it does not tell us what behaviour was checked

Branch Coverage Example

One business rule needs at least two tests

```
@Test
void appraisalForLowSalary() {
    employee.setMonthlySalary(9000);
    assertEquals(500, logic.calculateAppraisal(employee));
}

@Test
void appraisalForHighSalary() {
    employee.setMonthlySalary(12000);
    assertEquals(1000, logic.calculateAppraisal(employee));
}
```

- Both branches of the salary rule are now protected
- This is more meaningful than only increasing line coverage
- Branch coverage often reveals missing edge cases

Mutation Testing

Testing the tests

```
// Original
if (salary < 10000) return 500;

// Mutant
if (salary <= 10000) return 500;

// If tests still pass, the boundary was not protected.
```

- Mutation tools introduce small artificial bugs
- A good test suite should detect and kill most mutants
- Mutation score is often more revealing than line coverage
- Useful for critical business rules, but slower than normal tests

Static Analysis Tools

Automated checks for code quality signals

- Checkstyle: style and formatting conventions
- PMD: common bug patterns and code smells
- SpotBugs: bytecode-level bug detection
- SonarQube/SonarCloud: quality gates, duplications, complexity, vulnerabilities
- JaCoCo: Java code coverage

Static analysis is a reviewer assistant, not a replacement for review.

GitHub Actions: Continuous Metrics

Run tests and coverage on every push

```
name: Java CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
      - run: mvn test
      - run: mvn jacoco:report
```

- Metrics are more useful when they run automatically
- CI turns quality checks into a habit
- Pull requests can show whether a change increases risk

Quality Gates

Turning metrics into engineering policy

```
quality gate:
  coverage: >= 80%
  duplicated_lines: < 3%
  new_critical_issues: 0
  new_security_hotspots_reviewed: true
  max_method_complexity: inspect if > 10
```

- Quality gates are rules for accepting or rejecting changes
- Prefer gates on new code to avoid blocking legacy projects forever
- Use exceptions with explicit justification
- A gate should support learning, not blind punishment

Process Metrics

How the team changes software

Lead time

Time from idea or commit to production

Review time

Time a pull request waits for feedback

Change failure rate

Percentage of changes causing incidents

Defect escape rate

Bugs found after release instead of before

Process metrics are especially useful in project work and professional teams.

Metrics in Refactoring

Measure before and after, but explain the design reason

Before:

```
OrderService.confirm() complexity = 18  
duplicated validation blocks = 4  
tests covering payment failures = 0
```

After:

```
PaymentPolicy classes introduced  
confirm() complexity = 5  
branch tests for each policy
```

- Metrics help justify refactoring effort
- The goal is not lower numbers everywhere
- The real goal is simpler change, clearer responsibilities and safer evolution

Spec-Driven Metrics

When requirements become measurable artifacts

```
Requirement: Unknown report format returns an error
```

```
Acceptance test:
```

```
  given --format xml  
  when the command is executed  
  then the program exits with an error message
```

- Specification coverage: which requirements have tests?
- Traceability: which commits implement which tasks?
- Acceptance criteria pass rate: which expected behaviours are validated?
- Change impact: which modules and tests are affected by a spec change?

AI-Assisted Development Metrics

Measure control, not just productivity

- AI-generated code accepted without modification: useful but risky if too high
- Review findings in AI-assisted changes: correctness, security, style, architecture
- Test generation quality: assertions, edge cases, mutation score
- Prompt-to-spec traceability: can we explain why the code changed?

AI metrics should evaluate engineering control, not developer replacement.

Agentic Programming Metrics

When agents plan, edit, test and iterate

Task completion

Did the agent satisfy the acceptance criteria?

Tool success

Did builds, tests and checks run successfully?

Iteration count

How many repair cycles were needed?

Human interventions

Where did the agent need guidance or correction?

These metrics are useful for teaching students how to supervise coding agents.

A Practical Metrics Dashboard

Small teams need a small set of meaningful indicators

- Build status: passing or failing
- Test coverage and branch coverage on new code
- Complexity hotspots: top methods/classes to inspect
- Duplication percentage
- Open critical issues from static analysis
- Requirements or tasks without tests

Start with five metrics that change behaviour. Add more only when needed.

Mini-Lab

Measure, refactor, measure again

```
mvn test
mvn jacoco:report
# inspect target/site/jacoco/index.html
```

- Choose a small Java package from TAP or your assignment
- Run tests and collect coverage
- Identify one complex or low-cohesion class
- Apply one refactoring: Extract Method, Move Method or Strategy
- Run tests again and compare the metrics
- Write a short explanation: what improved, what did not, and why

Key Takeaways

How to use metrics professionally

- Metrics are indicators for discussion, not automatic judgments
- Complexity, coupling, cohesion and duplication help locate design risk
- Coverage must be interpreted with assertions, branches and behaviours
- CI/CD makes quality feedback continuous
- Spec-driven and AI-assisted development need traceability and validation metrics

Good metrics help us ask better engineering questions.